

PROTOCOLO DE COLABORACIÓN MULTI-AGENTE (V3.0)

Estado: Autorizado por el Supervisor (Usuario)

Ubicación sugerida: PROTOCOL.md (Raíz del proyecto)

I. EL BANCO DE MEMORIA (.context/)

Creados por el arquitecto (Gemini) fichero y los 5 archivos

Cada archivo responde a una pregunta crítica que el agente se hace al "despertar":

1. **projectBrief.md (¿Qué estamos haciendo?):** Contiene el objetivo macro y la visión del proyecto. (Revisión obligatoria en cada turno).
 2. **productContext.md (¿Cómo funciona esto?):** Explica la lógica de negocio y la experiencia de usuario esperada. (Revisión obligatoria en cada turno).
 3. **techContext.md (¿Con qué herramientas?):** Define el stack tecnológico, dependencias y arquitectura técnica. (Revisión obligatoria en cada turno).
 4. **activeContext.md (¿En qué estamos justo ahora?): Aquí se escribe el PLAN DETALLADO del Arquitecto.** Define los archivos a tocar, la lógica a implementar y las notas técnicas para el Programador. (Revisión obligatoria en cada turno).
 5. **progress.md (¿Qué hemos logrado?):** El historial de hitos y la lista de tareas pendientes. Se registran los turnos por número y su estado. (Revisión obligatoria en cada turno).
-

II. ROLES Y RESPONSABILIDADES

El Arquitecto (Gemini)

- **Misión:** Diseño técnico y planificación.
- **Capacidad:** Puede sugerir fragmentos de código, nombres de variables y lógica para guiar al Programador.
- **Restricción:** No toca archivos de código fuente.

El Programador (Claude Code)

- **Misión:** Implementación y documentación técnica.
- **Responsabilidad:** Notificar conflictos de UI o lógica mediante "**Notas Técnicas**" en los archivos de contexto.

III. FLUJO DEL JUEGO POR TURNOS (OPTIMIZADO)

1. **INICIO DE JORNADA:** El Agente en turno lee los 5 archivos de `.context/`.
2. **TURNNO: ARQUITECTO (Gemini)**
 - Lee los archivos dinámicos (`progress`, `activeContext`, `techContext`).
 - **PROPUESTA EN CHAT:** Presenta el plan general al Supervisor directamente en el chat.
 - **PAUSA:** Espera aprobación.
3. **VALIDACIÓN (Supervisor)**
 - Si a Usuario le gusta el plan: *"Me gusta, escríbelo en `activeContext.md`"*.
 - Gemini procede a documentar el plan detallado en el archivo.
4. **TURNNO: PROGRAMADOR (Claude Code)**
 - Lee el plan detallado en `activeContext.md`.
 - Ejecuta los cambios en el código.
 - Actualiza `progress.md` (checks) y deja el resumen del resultado en `activeContext.md`.

IV. PROTOCOLO DE CIERRE DE JORNADA

Cuando el Supervisor indique **"Cierre de jornada"**:

1. **Resumen Ejecutivo:** El **Programador** redacta un resumen de lo logrado durante el día.
2. **Registro:** Escribe este resumen en `progress.md` bajo `## Logros del Día [FECHA]`.
3. **Transferencia:** Deja una nota en `activeContext.md` con los pendientes específicos para el inicio de la siguiente jornada.
4. **Decisión de Commit:** Usuario valida si se realiza el commit / push manual.

V. REGLA DE REDUNDANCIA

Ante fallos de un modelo, el agente activo asume ambas funciones, manteniendo siempre la fase de **Propuesta en Chat** para validación del Supervisor.